

Arquitectura de Documentación Híbrida

"AI-First": Optimización de Contexto

RAG y Developer Experience

1. Fundamentos Clave (Insights de la investigación)

La estructuración de documentación de software contemporánea exige el abandono de los paradigmas tradicionales orientados exclusivamente a la lectura humana. La convergencia de la Generación Aumentada por Recuperación (RAG) y los Modelos de Lenguaje Grande (LLMs) requiere una arquitectura híbrida fundamentada en métricas de eficiencia de tokens, determinismo semántico y control estricto de la ventana de contexto. Las siguientes directivas innegociables sustentan esta arquitectura:

- **Determinismo Estructural y Resiliencia al "Chunking" (Optimización RAG):** El procesamiento vectorial de documentos extensos por parte de inteligencias artificiales depende de la fragmentación matemática (*chunking*) del texto en unidades predeterminadas.¹ Una jerarquía predecible y estricta de encabezados (H1 → H2 → H3) actúa como un mapa topológico que preserva las relaciones semánticas cuando el texto se divide.³ La eliminación absoluta de pronombres demostrativos ambiguos (ej. "esto", "aquello") y el uso exclusivo de sustantivos propios completos (ej. "El módulo de Autenticación") garantiza la independencia contextual; esto asegura que las coordenadas de los vectores de incrustación (*embeddings*) no se dispersen en el espacio latente, permitiendo que un fragmento aislado retenga su significado absoluto durante la recuperación.¹
- **Gestión Asimétrica del Contexto y Erradicación del "Impuesto de Tokens" (.mdc):** La inyección indiscriminada de reglas globales (como el obsoleto archivo `.cursorrules`) en la ventana de contexto genera una degradación inmediata del rendimiento algorítmico y aumenta exponencialmente los costos de inferencia.⁶ La transición obligatoria hacia el directorio `.cursor/rules/` con archivos Markdown Configurable (.mdc) permite una inyección condicional basada en metadatos YAML. Las reglas se activan dinámicamente mediante patrones globs (asociación automática de archivos), descripciones evaluadas por agentes autónomos, o invocaciones manuales (@), limitando la carga cognitiva de la IA estrictamente al dominio de la tarea en ejecución.⁶
- **Aislamiento Ortogonal del Conocimiento (Framework Diátaxis + Arc42):** Para mitigar la sobrecarga cognitiva humana y prevenir la dilución semántica en los LLMs, el conocimiento debe segregarse según su propósito.⁸ La metodología requiere la integración del framework Diátaxis (que clasifica la documentación en Tutoriales, Guías de Procedimiento, Referencia y Explicación) con recortes específicos de la plantilla Arc42 (para el modelado estático de la arquitectura).¹⁰ Esto evita que un agente de IA ingeste un tutorial paso a paso cuando requiere recuperar las políticas de infraestructura o de seguridad.⁸

- **Estandarización de Descubrimiento de Agentes (llms.txt):** El protocolo llms.txt opera como un manifiesto de indexación determinista para consumidores no humanos, equivalente funcional a robots.txt pero diseñado para agentes autónomos.¹² La provisión de este archivo en el directorio raíz garantiza que las herramientas de IA obtengan resúmenes ligeros, enlaces a especificaciones formales (OpenAPI/AsyncAPI) y acceso a versiones Markdown sin interfaces gráficas, eliminando la latencia y el desperdicio masivo de tokens asociado al análisis de estructuras HTML o diseños de interfaz de usuario.¹⁴
- **Trazabilidad Inmutable del Estado (Lightweight ADRs):** El código fuente y los manifiestos arquitectónicos principales no deben almacenar debates, alternativas descartadas o contexto histórico, ya que esto introduce ruido en las canalizaciones de incrustación y recuperación.² La adopción del estándar *Architecture Decision Records* (ADR) de Michael Nygard en formato Markdown simple garantiza que cada decisión tecnológica se capture en archivos secuenciales inmutables (ej. 0001-postgresql.md).¹⁷ Esto provee a los desarrolladores y a los sistemas de inferencia causalidad retrospectiva bajo demanda sin contaminar el contexto operativo actual.¹⁷

Para cuantificar el impacto algorítmico y humano de estas metodologías, la siguiente matriz técnica establece los vectores de optimización logrados por la arquitectura híbrida frente a las convenciones tradicionales:

Vector de Análisis	Estándar Tradicional (Pro-Humano)	Arquitectura Híbrida (Dev + AI)	Impacto Computacional / Cognitivo
Formato de Marcado	Confluence, HTML Crudo, Word ¹⁰	Markdown semántico puro (.md, .mdc) ⁶	Reducción drástica del desperdicio de tokens; eliminación total del ruido sintáctico generado por etiquetas de estilo y UI. ²
Modelado de Tablas	HTML avanzado, celdas combinadas o anidadas para estética ¹	Tablas Markdown estrictamente planas o listas de viñetas para datos densos ¹	Prevención del 100% de errores de análisis sintáctico (parsing) en LLMs; maximización de la precisión de extracción de datos estructurados. ¹

Activación de Reglas	Monolito de instrucciones globales (README.md masivo) ⁶	Modularidad asimétrica por subdominios vía globs y metadatos YAML ⁶	Eliminación del "impuesto de tokens". Prevención del olvido de instrucciones en el medio del prompt (Lost in the Middle anomaly). ⁶
Contexto de Referencia	Inferencia implícita basada en lectura secuencial ("esto se conecta aquí") ¹	Anclaje de datos con nombres propios absolutos y definiciones explícitas ¹	Incremento de precisión en la recuperación RAG mediante puntuaciones óptimas de similitud de coseno en bases de datos vectoriales. ¹
Historial de Decisiones	Conocimiento tribal esparcido en actas de reuniones y PRs cerrados ¹⁷	Archivos secuenciales monótonos (ADR) con formato de pirámide invertida ¹⁷	Comprensión autónoma retrospectiva. Curva de aprendizaje acelerada de semanas a <15 minutos para desarrolladores entrantes. ¹⁷
Evaluación de Código	Ausencia de ejemplos, suposición de competencia en el lenguaje ⁵	Bloques de código con etiquetado semántico XML (<example type="invalid">) ⁵	Habilitación de <i>In-Context Learning</i> de alto rendimiento. El modelo ajusta sus parámetros lógicos antes de generar la respuesta. ⁵

2. Árbol de Directorios (La Estructura)

El siguiente modelo de directorio define la infraestructura topológica de la documentación. Diseñado axiomáticamente bajo restricciones estáticas, este árbol opera simultáneamente

como un índice de abordaje sin fricción para humanos y como un grafo de ingesta vectorial hiperoptimizado para agentes de IA. La profundidad anidada se limita estrictamente a tres niveles para evitar la degradación en los recorridos de árbol (*tree traversals*) ejecutados por algoritmos de búsqueda.

/ | — llms.txt # Índice determinista de descubrimiento para LLMs (H1 principal, descripciones concisas).¹³ | — llms-full.txt # Agregación resuelta de contexto total (especificaciones + referencias completas).¹⁴ | — CONTEXT.md # Entrypoint híbrido: Mapeo estático de arquitectura (Arc42 subset) y reglas de negocio inmutables. | — README.md # Entrypoint Dev-First: Ejecución local, variables de entorno, y comandos de inicialización. | | — .cursor/ # Directorio de control de inferencia asimétrica y modelado de comportamiento.⁶ | | — rules/ # Repositorio versionado de archivos MDC (Markdown Configurable).⁷ | | — 000-core-standards.mdc # Reglas base innegociables. alwaysApply: true. Densidad semántica < 200 palabras.⁶ | | — 100-frontend-react.mdc # Contexto UI. globs: ["src/client/**/*.tsx", "src/components/**/*.ts"]. Límite < 500 palabras.⁶ | | — 200-backend-api.mdc # Contexto Servidor. globs: ["src/api/**/*.ts", "src/controllers/**/*.ts"].⁷ | | — 300-database-schemas.mdc # Contexto Persistencia. globs: ["prisma/**/*.prisma", "db/migrations/**/*.sql"]. | | — 800-stripe-payments.mdc # Regla bajo demanda. Sin globs. Se activa si el prompt del agente incluye "pagos" o "Stripe".⁶ | | — 900-testing-strategy.mdc # Regla bajo demanda. Se activa manualmente con @testing-strategy en el chat.⁶ | | — docs/ # Espacio de conocimiento particionado (Framework Diátaxis integrado con Arc42).⁸ | | — adr/ # Registros de Decisiones Arquitectónicas inmutables (Formato Michael Nygard).¹⁷ | | | — 0000-plantilla-adr.md # Plantilla estandarizada de pirámide invertida.¹⁷ | | | — 0001-adopcion-postgresql.md # Registro cronológico fundacional de la capa de datos.¹⁸ | | | — 0002-patron-event-sourcing.md # Registro del cambio de paradigma de comunicación entre servicios. | | | | — architecture/ # Documentación mutacional del estado del sistema (Arc42 Template modificado).²³ | | | | — 01-contexto-y-alcance.md # Modelado de límites del sistema (Interfaces C4 diagramadas en código).¹⁰ | | | | — 02-restricciones.md # Restricciones regulatorias, organizacionales y tecnológicas.²⁶ | | | | — 03-estrategia-despliegue.md # Topología de infraestructura estática (Kubernetes, AWS VPCs, etc.).⁸ | | | | — reference/ # Cuadrante Diátaxis: Especificaciones formales y exhaustivas independientes del contexto.¹¹ | | | | — api-rest-v1.yaml # Especificación OpenAPI (ingerida y formateada automáticamente por Redocly/Fern).³ | | | | — asyncapi-events.yaml # Especificación AsyncAPI para tópicos de Kafka y WebSockets.¹³ | | | | — how-to/ # Cuadrante Diátaxis: Guías de resolución de problemas orientadas a objetivos.⁹ | | | | — desarrollar-nuevo-endpoint.md # Instrucciones procedurales sin explicaciones teóricas subyacentes. | | | | — depurar-fugas-memoria.md # Protocolos de diagnóstico operativo estandarizados. | | | | — tutorials/ # Cuadrante Diátaxis: Aprendizaje dirigido para el onboarding de desarrolladores.¹¹ | | | | — mi-primer-microservicio.md # Recorrido guiado para la primera interacción con la base de código. | | | | — src/ # Código fuente de la aplicación estructuralmente aislado de la documentación. | | | | — client/ | | | | — api/ | | | | — package.json

Análisis Semántico de Ingesta y Recuperación Topológica

La estructura arbórea delineada opera bajo principios deterministas que suprimen la inferencia especulativa del modelo y optimizan la escalabilidad humana.

1. **Capa de Orientación y Metadatos Globales (/llms.txt, /llms-full.txt, /CONTEXT.md):** El alojamiento del estándar llms.txt en el nivel cero del directorio provee una interfaz de descubrimiento estandarizada.¹³ Cuando un LLM inicializa el análisis del repositorio, mapea instantáneamente el propósito del código y las dependencias documentales externas.¹² El archivo CONTEXT.md funciona como el manifiesto supremo: concentra las directivas empresariales y de dominio que deben inyectarse inherentemente en el *System Prompt* de cualquier agente, dictaminando las fronteras lógicas de la aplicación antes de que se ejecute la primera línea de análisis de código.²
2. **Motor de Restricción Asimétrica de Tokens (/cursor/rules/):** El abandono de un archivo centralizado a favor de archivos MDC versionados y compartimentalizados previene la contaminación de la memoria operativa de los modelos generativos.⁶ Mediante el uso de descriptores de coincidencia de patrones (*globs*), el sistema garantiza matemáticamente que las reglas semánticas de React (100-frontend-react.mdc) tengan cero probabilidad de colisionar en el espacio de tokens cuando el agente esté depurando lógica de persistencia en PostgreSQL (300-database-schemas.mdc).⁶ Los prefijos numéricos dictan la jerarquía de resolución de conflictos, y la categorización manual subyacente proporciona control de anulación explícito.²²
3. **Segregación de la Mutabilidad (Directorio docs/):** La arquitectura impone una bifurcación estricta entre el conocimiento inmutable y el mutable. El subdirectorio docs/adr/ contiene invariantes del pasado; un sistema RAG lee los ADRs exclusivamente para contextualizar el "porqué" histórico de una limitación tecnológica, previniendo refactorizaciones circulares sugeridas por la IA.¹⁷ Simultáneamente, docs/architecture/ representa el estado fotográfico actual de la arquitectura basado en la especificación Arc42.²³ La segregación Diátaxis (how-to/, tutorials/) evita que respuestas generativas sobre diseño de software se contaminen con lenguajes instructivos o comandos de consola irrelevantes.⁸

3. Plantilla Core (El Entregable)

La materialización del marco teórico híbrido requiere el despliegue de artefactos con codificación de alta densidad. Las siguientes plantillas constituyen el núcleo técnico de la implementación, garantizando la optimización del consumo de tokens y la erradicación de la fricción cognitiva para el usuario humano.

Artefacto 3.1: Índice de Agentes Autónomos (llms.txt)

Este artefacto sigue rigurosamente la especificación oficial para el descubrimiento algorítmico.¹³ Su estructura exige un H1 único, un *blockquote* que condensa el dominio ontológico en una sola oración, y secciones de Markdown estrictamente formatadas.¹³

Plataforma Central de Transacciones (Core Transaction Engine)

Sistema distribuido de procesamiento financiero de alta disponibilidad. La arquitectura emplea microservicios impulsados por eventos, persistencia en PostgreSQL con estricto cumplimiento ACID, y transmisión asíncrona mediante Apache Kafka. Este manifiesto expone las directivas de la arquitectura base.

Metadatos y Contexto del Dominio

-(CONTEXT.md): Definición inmutable de restricciones operativas, tecnológicas y de negocio.

- Protocolos de Inicialización Local: Instrucciones deterministas para la orquestación del entorno mediante contenedores Docker.

Estado Arquitectónico (Modelo Arc42)

-(docs/architecture/01-contexto-y-alcance.md): Mapeo estático C4 y topología de interfaces externas.

-(docs/architecture/02-restricciones.md): Normativas de seguridad (CISO), políticas de latencia y criptografía de datos.

Registro Histórico Inmutable

-(docs/adr/): Registro cronológico exhaustivo de decisiones tecnológicas fundamentales.

-(docs/adr/0001-adopcion-postgresql.md)

-(docs/adr/0002-patron-event-sourcing.md)

Especificaciones de Interfaz y Contratos

-(docs/reference/api-rest-v1.yaml): Contratos deterministas legibles por máquina para integraciones sincrónicas.

- AsyncAPI Events: Contratos semánticos para consumidores de colas de mensajería.

Opcional

-(docs/how-to/desarrollar-nuevo-endpoint.md): Procedimientos operativos estandarizados para la mutación del código.

Artefacto 3.2: Manifiesto Híbrido de Restricciones (CONTEXT.md)

Este documento constituye la columna vertebral del contexto. Utiliza lenguaje absoluto, suprime el ruido visual de tablas anidadas en favor de viñetas estructuradas, y define explícitamente acrónimos para anclar los vectores semánticos en las operaciones RAG.¹ Está diseñado como una versión ultracompactada de los capítulos más críticos del framework Arc42.¹⁰

Manifiesto Estructural y Restricciones

del Dominio (Core System)

1. Misión Operativa y Requisitos Funcionales

Axiomáticos

El Core System ejecuta procesamiento de transacciones financieras. La inmutabilidad de los registros de auditoría y la precisión del cálculo matemático priman absolutamente sobre las métricas de latencia de red.

- **Aislamiento Estricto de Datos:** Ningún servicio puede ejecutar consultas o mutaciones directas sobre los almacenes de datos (Databases) pertenecientes a otros servicios. La comunicación inter-dominio exige el uso de eventos asíncronos emitidos a través de Kafka.
- **Idempotencia Transaccional Obligatoria:** Todos los controladores HTTP que procesen mutaciones (POST, PUT, PATCH) deben requerir y validar un token de idempotencia en los encabezados (x-idempotency-key) para prevenir ejecuciones repetidas inducidas por fallas de red.

2. Restricciones de Negocio y Seguridad de la Información (Arc42 - Capítulo 2)

Estas políticas son de cumplimiento regulatorio estricto y no están sujetas a renegociación técnica.

- **Criptografía de Información de Identificación Personal (PII):** Todos los vectores de datos clasificados como PII deben ser cifrados en la capa de la aplicación antes de la persistencia (Cifrado en Reposo) utilizando el estándar AES-256-GCM.
- **Política de Borrado Lógico (Soft Deletes):** La ejecución del comando SQL DELETE está permanentemente prohibida para tablas transaccionales. Toda supresión de datos requiere la mutación exclusiva del registro de marca de tiempo deleted_at.
- **Autenticación de Máquina a Máquina (M2M):** La invocación de APIs internas demanda tokens JWT (JSON Web Tokens) asimétricos firmados mediante el algoritmo RS256, con una vigencia máxima innegociable de 15 minutos.

3. Pila Tecnológica Autorizada

La divergencia de estas tecnologías requiere la aprobación documentada mediante un nuevo archivo en el directorio de registros ADR (Architecture Decision Records).

- **Capa de Presentación (Frontend):** React 18, Next.js 14 (App Router), TailwindCSS v3.
- **Capa de Servicios (Backend):** Node.js v20 (LTS), Framework NestJS v10, TypeScript en modo estricto, Prisma ORM.
- **Persistencia y Caché:** PostgreSQL 15, Redis v7 (estrictamente para almacenamiento efímero y manejo de sesiones).
- **Orquestación de Mensajes:** Apache Kafka (Despliegue gestionado vía Confluent Cloud).

4. Convenciones Semánticas de Estructuración de

Código

- **Tipografía de Nombres:** Modelos de dominio y Entidades (PascalCase). Variables, métodos y controladores (camelCase). Constantes inmutables y variables de entorno configurables (UPPER_SNAKE_CASE).
- **Gestión Determinista de Excepciones:** Prohibida la instanciación de errores genéricos (throw new Error()). Toda anomalía lógica debe encapsularse en las clases derivadas DomainError o SystemError para garantizar la serialización JSON uniforme en los interceptores globales HTTP.
- **Paradigma de Documentación de Código:** El código debe manifestar su intención mediante nomenclaturas claras. Los comentarios en línea están estrictamente reservados para documentar algoritmos matemáticos complejos o parches a librerías de terceros; nunca deben duplicar la descripción de una función evidente (ej. // Crea un usuario sobre la función createUser() está prohibido).²⁸

Artefacto 3.3: Inyección Asimétrica de Reglas MDC (.cursor/rules/000-core-standards.mdc)

El núcleo del motor de IA reside en los archivos de configuración MDC. Este artefacto demuestra la sintaxis exacta del *frontmatter* YAML requerido para el control del alcance, seguido de directivas densas y el uso de bloques XML semánticos (<example>) para inducir un aprendizaje en contexto de alta fidelidad.⁶

YAML

description: Reglas fundacionales de arquitectura, tipado y estilo aplicables a la totalidad del repositorio base.

globs: "**/*.ts" "**/*.tsx" "**/*.js" "**/*.jsx"

alwaysApply: true

priority: 1

tags: core standards architecture typescript

version: 1.0.0

Estándares Core de Codificación y Arquitectura Limpia

Propósito y Activación de Regla

Esta directiva se aplica globalmente a todas las mutaciones del código fuente para garantizar la seguridad de tipos y la estricta observancia del paradigma de arquitectura en capas (Clean

Architecture).

Requisitos Innegociables del Análisis Estático

- Escribir código exclusivamente en TypeScript, forzando la directiva ``"strict": true`` en la configuración global de compilación.
- La declaración explícita de la palabra clave ``any`` está prohibida. En escenarios de resolución de tipos dinámicos en tiempo de ejecución, se debe emplear ``unknown`` respaldado por guardas de tipo exhaustivas (`*type narrowing*`).
- La lógica de negocio está confinada irrevocablemente a la capa de Casos de Uso (Use Cases) o Servicios de Dominio. Los Controladores HTTP están limitados a la validación de Entradas/Salidas (I/O) y a la manipulación del estado del protocolo de red.
- Todas las importaciones huérfanas deben eliminarse del AST (Abstract Syntax Tree) previo a la generación de la respuesta.

Bloques de Referencia de Modelos (Ejemplos Formativos)

<example>

// EJEMPLO VÁLIDO: Controlador anémico con delegación lógica y tipado seguro.

```
import Controller Post Body from '@nestjs/common';
```

```
import RegisterUserUseCase from '../use-cases/register-user.usecase';
```

```
import RegisterUserDto from '../dto/register-user.dto';
```

```
@Controller('users')
```

```
export class UserController
```

```
  constructor(private readonly registerUserUseCase: RegisterUserUseCase)
```

```
  @Post()
```

```
    async register(@Body() payload: RegisterUserDto)
```

```
    { return await this.registerUserUseCase.execute(payload);
```

```
  }
```

</example>

<example type="invalid">

// EJEMPLO INVÁLIDO: Contaminación lógica en el controlador y violación de tipos.

```
import Controller Post Body from '@nestjs/common';
```

```
import DatabaseService from '../database/db.service';
```

```
@Controller('users')
```

```
export class UserController
```

```
  constructor(private readonly db: DatabaseService)
```

```
  @Post()
```

```

    async register(@Body() payload: any) // Violación estructural: uso de 'any'.
    if (payload.age < 18) throw new Error("Edad insuficiente"); // Violación estructural: manejo de
    error primitivo.

    // Violación arquitectónica: inyección directa de DB en la capa de red.
    await this.db.users.insert(
      name: payload.name
      email: payload.email
    );
    return success: true ;
  }
</example>

```

Anatomía Computacional del Archivo MDC

- **Ingeniería de Metadatos YAML:** La conjunción del parámetro `alwaysApply: true` y una calificación de `priority: 1` posiciona a este archivo como la base axiomática del comportamiento del agente.⁶ El uso del array de metadatos `tags` optimiza el enrutamiento interno de categorización de IA.²² Para mantener la salud presupuestaria de los tokens y prevenir la saturación de la memoria (*Token Tax*), el contenido descriptivo general se comprime obligatoriamente por debajo del umbral de 200 palabras.⁶
- **Anclajes XML Semánticos:** Los Modelos de Lenguaje Grande responden matemáticamente mejor a restricciones estructuradas mediante etiquetas XML (como `<example type="invalid">`). Esta estructuración provee un vector de contranarrativa (*negative constraint*), instruyendo explícitamente a la red neuronal sobre los patrones que debe penalizar durante la inferencia secuencial de código.²²

Artefacto 3.4: Inyección Dinámica Orientada al Dominio (.cursor/rules/100-frontend-react.mdc)

A diferencia del manifiesto global (Artefacto 3.3), este bloque demuestra la eficiencia algorítmica de la activación condicional. Solo se inyectará en la matriz de tokens si el sistema de archivos actual coincide con el patrón *glob*.

```

YAML
---
description: Reglas de renderizado de UI, gestión de estado y convenciones de componentes en

```

```
React.  
globs: "src/client/**/*.tsx" "src/client/**/*.ts" "src/components/**/*.tsx"  
alwaysApply: false  
priority: 2  
tags: frontend react ui components  
---
```

```
# Directivas de Desarrollo Frontend (React / Next.js)
```

```
## Requisitos Innegociables
```

- Renderizar exclusivamente Componentes Funcionales (Functional Components) declarados mediante funciones de flecha (`const`). Queda prohibida la sintaxis antigua de componentes basados en clases.
- Forzar la definición exhaustiva de `Props` mediante interfaces de TypeScript. El uso de `React.FC` está desaconsejado a favor del tipado directo de los argumentos.
- Los módulos deben utilizar exportaciones con nombre (Named Exports). Se prohíben categóricamente las exportaciones por defecto (`export default`) para garantizar una refactorización segura y búsquedas deterministas de símbolos en el AST.
- La extracción de datos (Data Fetching) en componentes de cliente debe orquestarse a través del hook de la librería TanStack Query; prohibido el uso directo de `useEffect` para el manejo de peticiones de red asíncronas.

4. Guía Rápida de Mantenimiento y Flujos (Workflows)

La viabilidad a largo plazo de cualquier arquitectura de conocimiento híbrido depende de protocolos operativos estandarizados que impidan la "entropía documental" (el proceso mediante el cual el código diverge silenciosamente de sus reglas operativas, causando alucinaciones masivas en los modelos de lenguaje).⁷ Los siguientes flujos de trabajo dictan la operatividad obligatoria para todos los contribuyentes.

Flujo Operativo 1: Desarrollo Cotidiano (Features, Bugs y Fixes)

Para prevenir la degradación de los artefactos nucleares, el desarrollo de ciclos cortos no debe invocar modificaciones a la documentación core (CONTEXT.md o reglas globales MDC).

1. **Activación de Contexto de Ruta:** El desarrollador aísla el archivo objetivo. Si el objetivo es `src/api/users.controller.ts`, el motor IDE (por ejemplo, Cursor) autodetecta el patrón `globs` y carga silenciosamente `.cursor/rules/200-backend-api.mdc` en el contexto latente.⁶ No se requiere carga manual.
2. **Inferencia Específica del Dominio:** El desarrollador suministra una directiva natural (ej. "Refactorizar método POST para usar DTO de creación"). El LLM hereda de forma automática e invisible las restricciones de *Clean Architecture* e *Idempotencia* definidas en la arquitectura híbrida.⁶
3. **Trazabilidad Continua:** Tras la resolución del Bug o culminación del Feature, no se edita documentación narrativa extensa. En cambio, se delega en agentes autónomos la

extracción de resúmenes de *commits* semánticos (ej. feat(api): implementacion de idempotencia en POST) y la estructuración dinámica de *Changelogs* o anotaciones de *Release Notes*.²⁹ Las soluciones a Bugs complejos deben alimentarse a un prompt estandarizado (ej. "Resume esta transcripción/fix técnico y extrae pasos de troubleshooting") para que la herramienta de documentación de IA (ej. Notion AI) lo incruste en la sección de guías Diátaxis sin intervención manual pesada.²⁹

Flujo Operativo 2: Alteración Topológica mediante ADRs (Architecture Decision Records)

Cualquier cambio que altere el estado de la pila tecnológica, los patrones de diseño (ej. transicionar de REST a GraphQL), o la infraestructura base, debe registrarse por el patrón de registro inmutable ADR de Nygard. Modificar subrepticamente el archivo CONTEXT.md sin un ADR subyacente destruye la memoria de la inteligencia artificial sobre el "porqué" de la arquitectura actual.¹⁷

Procedimiento de Alteración:

1. **Creación del Registro:** Instanciar un nuevo archivo dentro de docs/adr/ respetando el índice monótono secuencial (ej. 0003-migracion-a-graphql.md).¹⁷
2. **Redacción de Pirámide Invertida:** Llenar el documento forzando los cinco nodos obligatorios: *Title*, *Status*, *Context*, *Decision*, y *Consequences*.¹⁹ El lenguaje debe ser quirúrgico, objetivo, y detallar obligatoriamente las consecuencias negativas o de costo técnico (Trade-offs).¹⁸
3. **Actualización del Manifiesto Core:** Únicamente después de que el *Status* del ADR cambie a Aceptado (Accepted), se actualiza el CONTEXT.md y, si procede, se reescribe un archivo de regla .mdc para entrenar a los agentes sobre la nueva decisión.⁷

Plantilla Base ADR (0000-plantilla-adr.md):

ADR-XXXX:

Status

Contexto

.

Decisión

.

Consecuencias

[Listado riguroso e imparcial de los efectos residuales posteriores a la ejecución de la decisión].

- (+) Reducción drástica del ancho de banda de red en clientes móviles.
- (+) Consolidación de peticiones de red múltiples en un único endpoint de orquestación.
- (-) Aumento de la complejidad operativa en la capa de caché. Caché HTTP a nivel de CDN quedará invalidado y se requerirá caché a nivel de resolver de Apollo.
- (-) Requiere instanciar el archivo .cursor/rules/400-graphql-resolvers.mdc para evitar que

el LLM alucine construcciones RESTful.

Flujo Operativo 3: Auditoría Cognitiva y Mantenimiento de Tokens (Context Pruning)

Los sistemas documentales impulsados por RAG (Retrieval-Augmented Generation) exhiben propensión a la inflación estática de contexto: acumulan exceso de ruido o documentos irrelevantes que diluyen los resultados de la búsqueda híbrida vectorial.² Las siguientes métricas de saneamiento son mandatorias:

1. **Poda de Reglas de Agente (MDC Pruning):** Bimestralmente, revisar exhaustivamente el tamaño de los archivos `.cursor/rules/*.mdc`. Todo archivo configurado como `alwaysApply: true` que vulnere el umbral de las 200 palabras debe fraccionarse modularmente para contener el *Token Tax*.⁶ Reemplazar bloques de texto iterativos por referencias enlazadas a los manuales más profundos dentro de `docs/` o externalizarlos a globs altamente específicos.⁷
2. **Sincronización Autónoma de Índices RAG (llms.txt):** El mapeo para consumo de agentes nunca debe depender de actualizaciones manuales continuas.¹⁴ Integrar herramientas automatizadas de CI/CD (Integración y Despliegue Continuo) —como Fern o Redocly— que recompilen `llms.txt` y `llms-full.txt` dinámicamente con base en las especificaciones OpenAPI vigentes en cada commit de la rama principal.³ Esto garantiza que las sugerencias de la IA utilicen esquemas invariablemente sincronizados, erradicando los tiempos muertos asociados con endpoints alucinados.¹⁶
3. **Evaluación de Trazabilidad Cruzada de Markdown:** Ejecutar scripts de validación de contexto para probar agresivamente la estructura frente a varios modelos fundacionales (ej. Claude 3.5 Sonnet, GPT-4o). Este escaneo simulado expande `llms.txt` para corroborar que ninguna celda combinada de tabla o etiqueta HTML residual destruya el analizador (*parser*) sintáctico del modelo, asegurando respuestas deterministas sobre toda la base de código.¹

Esta estructura híbrida no constituye meramente una convención de escritura; es un motor semántico operativo. Al alinear el modelado arquitectónico estructurado humano (Diátaxis, Arc42, ADRs) con la inyección dinámica basada en dominios para sistemas de inferencia computacional (MDC, `llms.txt`), se suprime el estrangulamiento de los límites de contexto y se garantiza la resiliencia a largo plazo de la plataforma.

Fuentes citadas

1. Writing for AI. Writing Guide for LLM and RAG | by Ragina Jeon | Medium, acceso: mayo 28, 2026, <https://lyingdragon.medium.com/writing-for-ai-caab3344f279>
2. I Built a RAG System That Knows My Entire Codebase (Here's How It Works). Stop searching docs for hours. Ask one question and get the exact answer from your own data. | by Satyadev Komaragiri | Medium, acceso: mayo 28, 2026, <https://medium.com/@satyadevk/i-built-a-rag-system-that-knows-my-entire-codebase-heres-how-it-works-stop-searching-docs-for-494b13b48f17>

3. How to optimize your docs for LLMs - Redocly, acceso: mayo 28, 2026, <https://redocly.com/blog/optimizations-to-make-to-your-docs-for-llms>
4. How to optimize your documentation for AI (without breaking it for humans) - GitBook, acceso: mayo 28, 2026, <https://www.gitbook.com/blog/ai-docs-optimization-tips>
5. Optimizing technical documentations for LLMs - DEV Community, acceso: mayo 28, 2026, <https://dev.to/joshtom/optimizing-technical-documentations-for-llms-4bcd>
6. Cursor Rules: Complete .mdc Guide & 15 Templates (2026), acceso: mayo 28, 2026, <https://www.vibecodingacademy.ai/blog/cursor-rules-complete-guide>
7. Mastering .mdc Files in Cursor: Best Practices | by Venkat | Medium, acceso: mayo 28, 2026, <https://medium.com/@ror.venkat/mastering-mdc-files-in-cursor-best-practices-f535e670f651>
8. Infrastructure in architectural documentation - INNOQ, acceso: mayo 28, 2026, <https://www.innoq.com/en/articles/2025/05/infrastructure-in-architectural-documentation/>
9. Ask HN: How do you organize software documentation at work? - Hacker News, acceso: mayo 28, 2026, <https://news.ycombinator.com/item?id=39370226>
10. Diataxis - Java & Moi, acceso: mayo 28, 2026, <https://javaetmoi.com/tag/diataxis/>
11. almereyda/awesome-starred: A curated list of my GitHub stars!, acceso: mayo 28, 2026, <https://github.com/almereyda/awesome-starred>
12. Making Your Documentation AI-Friendly: A Practical Guide - DEV Community, acceso: mayo 28, 2026, https://dev.to/sudo_overflow/making-your-documentation-ai-friendly-a-practical-guide-2h1f
13. llms.txt - Mintlify, acceso: mayo 28, 2026, <https://www.mintlify.com/docs/ai/llmstxt>
14. llms.txt and llms-full.txt | Fern Documentation, acceso: mayo 28, 2026, <https://buildwithfern.com/learn/docs/ai-features/llms-txt>
15. Best practices for converting documentation to AI-friendly format? : r/ChatGPTCoding, acceso: mayo 28, 2026, https://www.reddit.com/r/ChatGPTCoding/comments/1hg8m52/best_practices_for_converting_documentation_to/
16. API Docs for AI Agents: llms.txt Guide May 2026 | Fern, acceso: mayo 28, 2026, <https://buildwithfern.com/post/optimizing-api-docs-ai-agents-llms-txt-guide>
17. Architecture Decision Record - Martin Fowler, acceso: mayo 28, 2026, <https://martinfowler.com/bliki/ArchitectureDecisionRecord.html>
18. ADR Templates – Architecture Decision Record Formats - ReflectRally, acceso: mayo 28, 2026, <https://reflectrally.com/adr-templates/>
19. peter-evans/lightweight-architecture-decision-records - GitHub, acceso: mayo 28, 2026, <https://github.com/peter-evans/lightweight-architecture-decision-records>
20. Lightweight Architecture Documentation with ADRs - Production Ready Blog, acceso: mayo 28, 2026, <https://www.production-ready.de/2023/12/28/lightweight-architecture-document>

[ation-adr-en.html](#)

21. Architecture decision record (ADR) - GitHub, acceso: mayo 28, 2026, <https://github.com/architecture-decision-record/architecture-decision-record>
22. Optimal structure for .mdc rules files - Discussions - Cursor - Community Forum, acceso: mayo 28, 2026, <https://forum.cursor.com/t/optimal-structure-for-mdc-rules-files/52260>
23. Arc42 Template in Markdown.md - GitHub, acceso: mayo 28, 2026, <https://github.com/NetworkedAssets/arc42-in-markdown-template/blob/master/2.%20Single%20File%2FArc42%20Template%20in%20Markdown.md>
24. arc42 - the template for software architecture documentation and communication - GitHub, acceso: mayo 28, 2026, <https://github.com/arc42/arc42-template>
25. MkDocs based arc42 Architecture Template - GitHub, acceso: mayo 28, 2026, <https://github.com/tassilosmola/architecture-mkdocs-template>
26. arc42-in-markdown-template/1. Files Tree/About arc42.md at master - GitHub, acceso: mayo 28, 2026, <https://github.com/NetworkedAssets/arc42-in-markdown-template/blob/master/1.%20Files%20Tree/About%20arc42.md>
27. llms-txt: The /llms.txt file, acceso: mayo 28, 2026, <https://llmstxt.org/>
28. Documenting code with AI, explaining every single symbol of a codebase - Reddit, acceso: mayo 28, 2026, https://www.reddit.com/r/programming/comments/1ajrdd5/documenting_code_with_ai_explaining_every_single/
29. AI-Powered Documentation: The Secret to Efficient Technical Writing - 8th Light, acceso: mayo 28, 2026, <https://8thlight.com/insights/ai-powered-documentation-the-secret-to-efficient-technical-writing>
30. Generate documentation | AI Assistant - JetBrains, acceso: mayo 28, 2026, <https://www.jetbrains.com/help/ai-assistant/generate-documentation-with-ai.html>